

Reviewer Pack — Minimal Index of p-Adic Hodge Structures and DPLL Search Tre...

Entry 255b8e500778 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 08:59:39 UTC

Minimal Index of p-Adic Hodge Structures and DPLL Search Tree Diameter

Entry ID: 255b8e500778 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 08:59:39 UTC

1. Conjecture

Field A (mathematical branch): p-adic Hodge Theory **Field B** (complexity object): DPLL Search Trees

Statement:

For every instance of an NP-complete problem, the minimal index of its associated p-adic Hodge structure ($mH(G)$) is linearly correlated with its DPLL search tree diameter (d), such that $mH(G) = \Theta(d^{1/4})$.

Rationale (proposer's reasoning):

p-adic Hodge theory provides a framework for studying arithmetic properties of algebraic varieties, and the minimal index could capture intrinsic arithmetic complexity. The DPLL search tree diameter measures the depth of the most complex path in the search space, which may be related to the arithmetic complexity of the problem.

Taxonomy category: p-adic_Hodge_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 80bcc69d0d72dfa6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an NP-complete problem instance, if the ratio of the minimal index of its associated p-adic Hodge structure ($mH(G)$) to the DPLL search tree diameter (d) is within $\pm 10\%$ of $1/4$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'p-adic Hodge Theory' AND 'DPLL search tree' AND correlation - 'mH(G)' AND 'DPLL search tree diameter' AND linear relationship - 'p-adic Hodge structure' AND ' $\theta(d^{1/4})$ ' AND DPLL search trees

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n):
        clauses = []
        for _ in range(3 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            if random.choice([True, False]):
                clause[0], clause[1] = -clause[0], -clause[1]
            clauses.append(clause)
        return clauses

    def dpll_search_tree_diameter(instance):
        n = len(instance)

        def dfs(model, depth):
            if all([any([model[abs(l) - 1] == l // abs(l) for l in clause]) for clause in instance]):
                return depth
            unsatisfied_clauses = [clause for clause in instance if not any([model[abs(l) - 1] == l // abs(l) for l in clause])]
            if not unsatisfied_clauses:
                return float('inf')
            literal = random.choice(unsatisfied_clauses[0])
            new_model_true = model[:]
            new_model_true[abs(literal) - 1] = literal // abs(literal)
            new_model_false = model[:]
            new_model_false[abs(literal) - 1] = -(literal // abs(literal))
            return min(dfs(new_model_true, depth + 1), dfs(new_model_false, depth + 1))

        initial_model = [0] * n
```

```

    return dfs(initial_model, 0)

def p_adic_hodge_structure(instance):
    # Simplified mapping for demonstration purposes
    return len(instance) ** (1/4)

instance = generate_instance(20)
d = dpll_search_tree_diameter(instance)
mH_G = p_adic_hodge_structure(instance)

if d == 0:
    return {
        "metric_name": "mH(G)/d^(1/4)",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": 20,
        "conjecture_holds": False,
        "counterexample": "DPLL search tree diameter is zero"
    }

ratio = mH_G / (d ** (1/4))
return {
    "metric_name": "mH(G)/d^(1/4)",
    "metric_value": ratio,
    "instances_tested": 1,
    "n_max": 20,
    "conjecture_holds": abs(ratio - Fraction(1, 4)) <= Fraction(1, 10),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_ratio)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mH(G)/d^(1/4) ratio outside ±10% of 1/4\" first={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a2e3cdf.py", line 82, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a2e3cdf.py", line 54, in run_trial
    d = dpll_search_tree_diameter(instance)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a2e3cdf.py", line 47, in dpll_search_t
    return dfs(initial_model, 0)
           ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a2e3cdf.py", line 44, in dfs
    return min(dfs(new_model_true, depth + 1), dfs(new_model_false, depth + 1))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a2e3cdf.py", line 44, in dfs
    return min(dfs(new_model_true, depth + 1), dfs(new_model_false, depth + 1))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a2e3cdf.py", line 44, in dfs
    return min(dfs(new_model_true, depth + 1), dfs(new_model_false, depth + 1))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 992 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a2e3cdf.py", line 39, in dfs
    literal = random.choice(unsatisfied_clauses[0])
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 348, in choice
    return seq[self._randbelow(len(seq))]
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data that could be used to evaluate the conjecture.
| next: Re-run the test with increased recursion limits or use an iterative approach to avoid the crash.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20288
2	preregistration	ollama_remote	glm4:latest	0	0	16366
3	novelty	ollama_remote	glm4:latest	0	0	15494

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	15371
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15119
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21500
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11247
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27699
9	judge	ollama_remote	glm4:latest	0	0	11580

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154664 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/255b8e500778.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/255b8e500778.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/255b8e500778.tar.gz` (if generated)